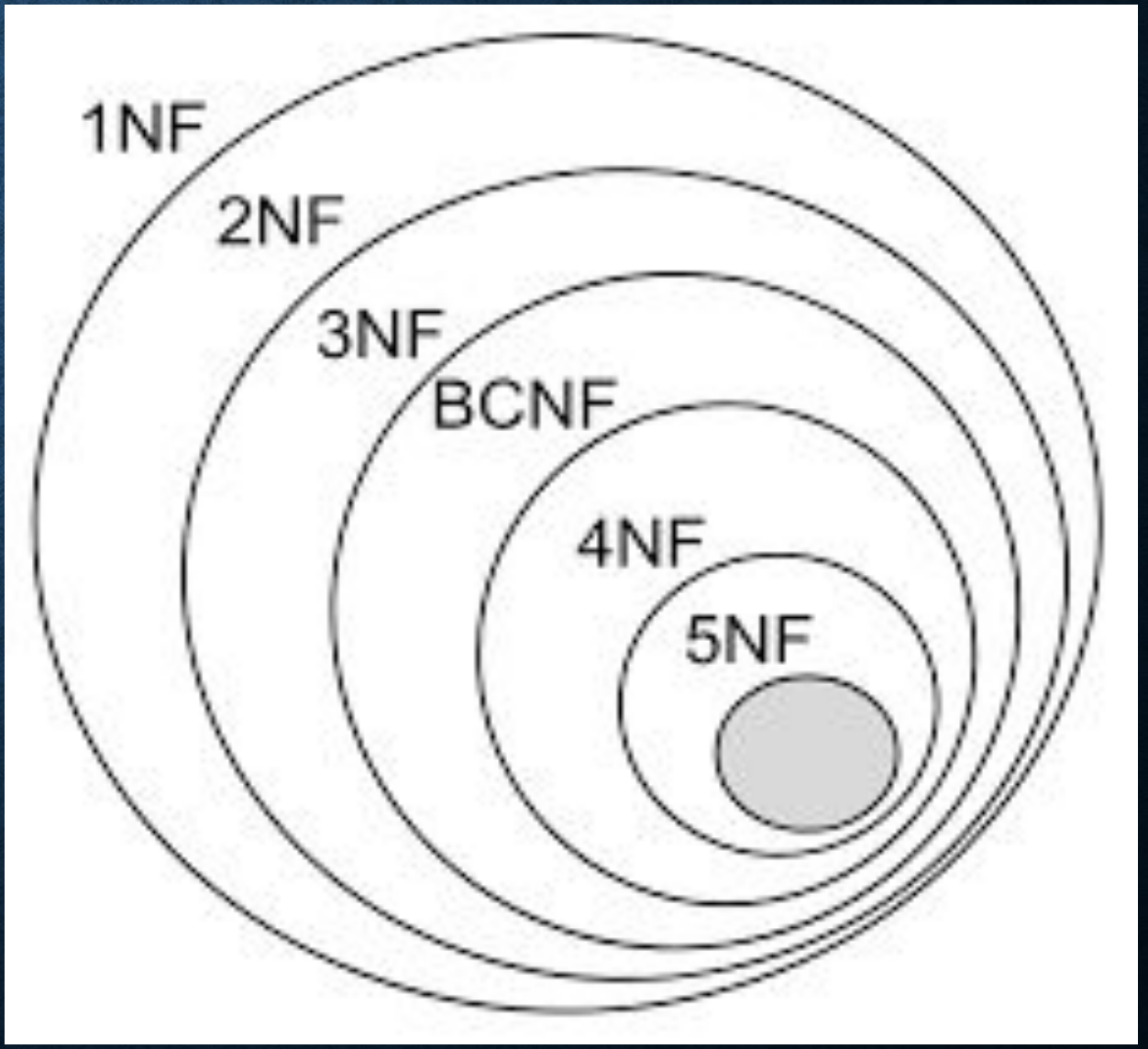
# NORMILIZATION

K Krishna priya

CSE ,IIIT Nuzvid



# NORMALIZATION OF DBMS

- In database management systems (DBMS), normal forms are a series of guidelines that help to ensure that the design of a database is efficient, organized, and free from data anomalies.
- Normalization also helps to organize the data in the database. It is a multi-step process that sets the data into tabular form and removes the duplicated data from the relational tables.
- There are several levels of normalization, each with its own set of guidelines, known as normal forms.

# NORMAL FORMS IN DBMS

- **Normalization** is the process of minimizing **redundancy** from a relation or set of relations. Redundancy in relation may cause insertion, deletion, and update anomalies.
- So, it helps to minimize the redundancy in relations.
- **Normal forms** are used to eliminate or reduce redundancy in database tables.
- Normalization organizes the columns and tables of a database to ensure that database integrity constraints properly execute their dependencies. It is a systematic technique of decomposing tables to eliminate data redundancy (repetition) and undesirable characteristics like Insertion, Update, and Deletion anomalies.

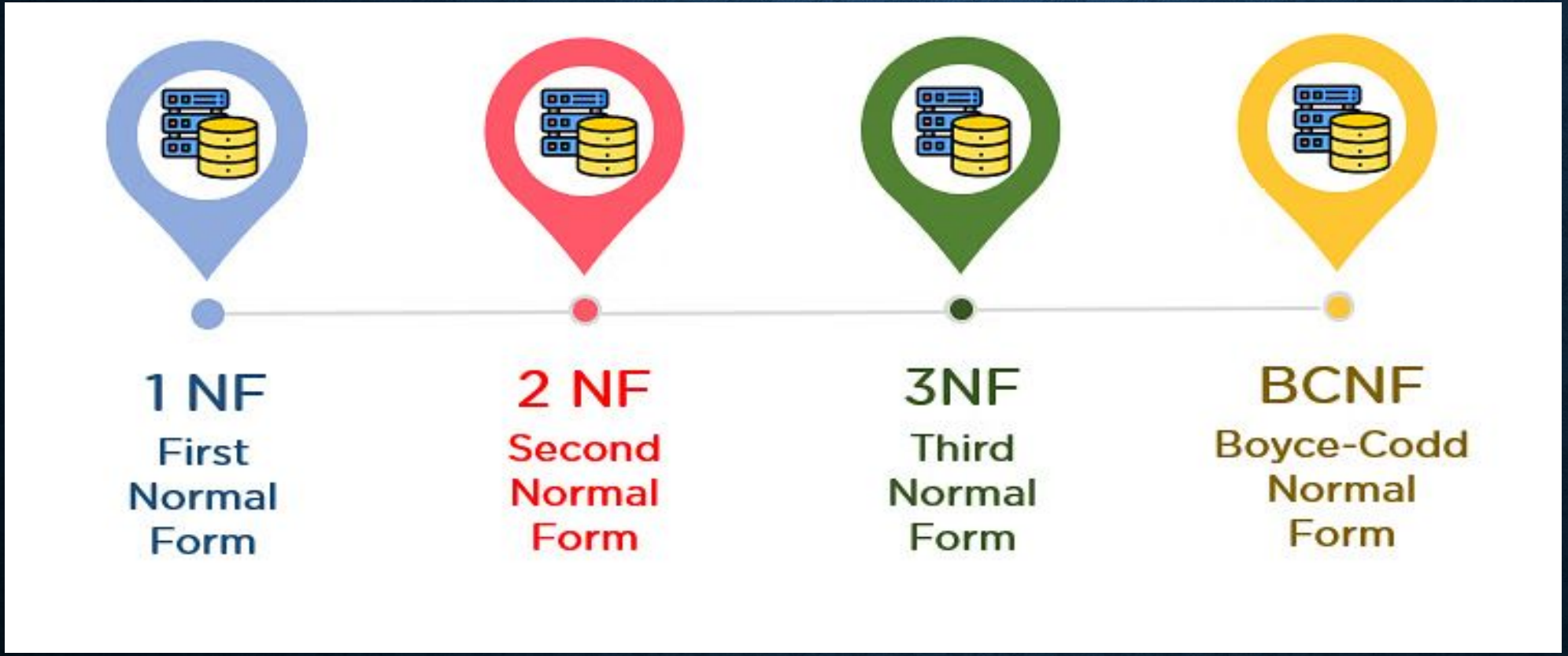
# WHY DO WE NEED NORMALIZATION?

- As we have discussed above, normalization is used to reduce data redundancy. It provides a method to remove the following anomalies from the database and bring it to a more consistent state:
- A database anomaly is a flaw in the database that occurs because of poor planning and redundancy.
- **Insertion anomalies**: This occurs when we are not able to insert data into a database because some attributes may be missing at the time of insertion.
- **Updation anomalies**: This occurs when the same data items are repeated with the same values and are not linked to each other.
- **Deletion anomalies**: This occurs when deleting one part of the data deletes the other necessary information from the database.

# SOME FACTS ABOUT DATABASE NORMALIZATION

- The words normalization and normal form refer to the structure of a
- database.
- Normalization was developed by IBM researcher E.F. Codd In the 1970s.
- Normalization increases the clarity in organizing data in Database.
- Normalization of a Database is achieved by following a set of rules
- called ‘forms’ in creating the database.

- **Normal Forms**
- There are four types of normal forms that are usually used in relational databases as you can see in the following figure:



# FIRST NORMAL FORM (1NF)

- A relation is in 1NF if every attribute is a single-valued attribute or it does not contain any multi-valued or composite attribute, i.e., every attribute is an atomic attribute.
- If there is a composite or multi-valued attribute, it violates the 1NF.
- To solve this, we can create a new row for each of the values of the multi-valued attribute to convert the table into the 1NF.
- Let's take an example of a relational table <EmployeeDetail> that contains the details of the employees of the company.

| Employee Code | Employee Name | Employee Phone Number |
|---|---|---|
| 101 | John | 98765623,998234123 |
| 101 | John | 89023467 |
| 102 | Ryan | 76213908 |
| 103 | Stephanie | 98132452 |

Here, the Employee Phone Number is a multi-valued attribute. So, this relation is not in

- To convert this table into 1NF, we make new rows with each Employee Phone Number as a new row as shown below:

| Employee Code | Employee Name | Employee Phone Number |
|---|---|---|
| 101 | John | 998234123 |
| 101 | John | 98765623 |
| 101 | John | 89023467 |
| 102 | Ryan | 76213908 |
| 103 | Stephanie | 98132452 |

# PARTIAL DEPENDENCY

- To be in second normal form, a relation must be in first normal form and relation
- must not contain any partial dependency.
- A relation is in 2NF if it has No Partial Dependency, i.e., no non-prime attribute (attributes which are not part of any candidate key) is dependent on any proper subset of any candidate key of the table.
- Partial Dependency – If the proper subset of candidate key determines non-prime
- attribute, it is called partial dependency.

# SECOND NORMAL FORM (2NF)

- The normalization of 1NF relations to 2NF involves the elimination of partial dependencies. A partial dependency in DBMS exists when any non-prime attributes, i.e., an attribute not a part of the candidate key, is not fully functionally dependent on one of the candidate keys.
- For a relational table to be in second normal form, it must satisfy the following rules:
- The table must be in first normal form.
- It must not contain any partial dependency, i.e., all non-prime attributes are fully functionally dependent on the primary key.
- If a partial dependency exists, we can divide the table to remove the partially dependent attributes and move them to some other table where they fit in well.
- Let us take an example of the following <EmployeeProjectDetail> table to understand what is partial dependency and how to normalize the table to the second normal form:

| Employee Code | Project ID | Employee Name | Project Name |
|---|---|---|---|
| 101 | P03 | John | Project103 |
| 101 | P01 | John | Project101 |
| 102 | P04 | Ryan | Project104 |
| 103 | P02 | Stephanie | Project102 |

In the above table, the prime attributes of the table are Employee Code and Project ID. We have partial dependencies in this table because Employee Name can be determined by Employee Code and Project Name can be determined by Project ID. Thus, the above relational table violates the rule of 2NF.

The prime attributes in DBMS are those which are part of one or more candidate keys.

To remove partial dependencies from this table and normalize it into second normal form, we can decompose the <EmployeeProjectDetail> table into the following three tables:

**<EmployeeDetail>**

| Employee Code | Employee Name |
|---|---|
| 101 | John |
| 101 | John |
| 102 | Ryan |
| 103 | Stephanie |

**EmployeeProject**

| Employee Code | Project ID |
|---|---|
| 101 | P03 |
| 101 | P01 |
| 102 | P04 |
| 103 | P02 |

**ProjectDetail**

| Project ID | Project Name |
|---|---|
| P03 | Project103 |
| P01 | Project101 |
| P04 | Project104 |
| P02 | Project102 |

- Thus, we've converted the <EmployeeProjectDetail> table into 2NF by decomposing it into <EmployeeDetail>, <ProjectDetail> and <EmployeeProject> tables.
- As you can see, the above tables satisfy the following two rules of 2NF as they are in 1NF and every non-prime attribute is fully dependent on the primary key.

The relations in 2NF are clearly less redundant than relations in 1NF.

- However, the decomposed relations may still suffer from one or more anomalies due to the transitive dependency. We will remove the transitive dependencies in the Third Normal Form.

# •Example 2nd Normal Form

| STUD_NO | COURSE_NO |
|---|---|
| 1 | C1 |
| 2 | C2 |
| 3 | C4 |
| 4 | C3 |
| 5 | C1 |

| COURSE_NO | COURSE_FEE |
|---|---|
| C1 | 1000 |
| C2 | 1500 |
| C3 | 1300 |
| C4 | 2000 |
| C5 | 2000 |

- No Non-prime attribute such as COURSE_FEE is dependent on a proper subset of
- the candidate key i e {STUD_NO, COURSE_NO} so no partial dependency and so these relations are in 2NF.
- 2NF tries to reduce the redundant data getting stored in memory. For instance, if
- there are 100 students taking C1 course, we don't need to store its Fee as 1000 for
- all the 100 records, instead, once we can store it in the second table as the course
- fee for C1 is 1000.

# THIRD NORMAL FORM (3NF)

- The normalization of 2NF relations to 3NF involves the elimination of transitive dependencies in DBMS.
- A functional dependency X -> Z is said to be transitive if the following three functional dependencies hold:
- X -> Y , Y does not -> X ,Y -> Z
- **For a relational table to be in third normal form, it must satisfy the following rules:**

1. **The table must be in the second normal form.**
2. **No non-prime attribute is transitively dependent on the primary key.**

- **For each functional dependency X -> Z at least one of the following conditions hold:**
- **X is a super key of the table.**
- **Z is a prime attribute of the table.**
- A relation is in 3NF if the relation is in 1NF, 2NF and all determinants of non-key attributes are candidate keys That is, for any functional dependency: X -> Y, where Y is a non-key attribute (or a set of non-key attributes), X is a candidate key.
- If a transitive dependency exists, we can divide the table to remove the transitively dependent attributes and place them to a new table along with a copy of the determinant.

- **Example 1:** In relation STUDENT given here…
- FD set: {STUD_NO -> STUD_NAME, STUD_NO -> STUD_STATE, STUD_STATE -> STUD_COUNTRY, STUD_NO -> STUD_AGE }
- Candidate Key: {STUD_NO}
- For this relation in table , STUD_NO -> STUD_STATE and STUD_STATE -> STUD_COUNTRY are true.
- So STUD_COUNTRY is transitively dependent on STUD_NO. It violates the third normal form.
- To convert it in third normal form, we will decompose the relation
- STUDENT (Stud_no, Stud_name, Stud_phone, Stud_state, Stud_age)
- STATE_COUNTRY (State, Country)

Let us take an example of the following <EmployeeDetail> table to understand what is transitive dependency and how to normalize the table to the third normal form:

**<EmployeeDetail>**

| Employee Code | Employee Name | Employee Zipcode | Employee City |
|---|---|---|---|
| 101 | John | 110033 | Model Town |
| 101 | John | 110044 | Badarpur |
| 102 | Ryan | 110028 | Naraina |
| 103 | Stephanie | 110064 | Hari Nagar |

The above table is not in 3NF because it has Employee Code -> Employee City transitive dependency because:

- Employee Code -> Employee Zipcode
- Employee Zipcode -> Employee City

Also, Employee Zipcode is not a super key and Employee City is not a prime attribute.

To remove transitive dependency from this table and normalize it into the third normal form, we can decompose the <EmployeeDetail> table into the following two tables:

**<EmployeeDetail>**

| Employee Code | Employee Name | Employee Zipcode |
|---|---|---|
| 101 | John | 110033 |
| 101 | John | 110044 |
| 102 | Ryan | 110028 |
| 103 | Stephanie | 110064 |

**<EmployeeLocation>**

| Employee Zipcode | Employee City |
|---|---|
| 110033 | Model Town |
| 110044 | Badarpur |
| 110028 | Naraina |
| 110064 | Hari Nagar |

Thus, we've converted the <EmployeeDetail> table into 3NF by decomposing it into <EmployeeDetail> and <EmployeeLocation> tables as they are now in 2NF and they don't have any transitive dependency.

- The 2NF and 3NF impose some extra conditions on dependencies on candidate keys and remove redundancy caused by that.

# CONT…

- However, there may still exist some dependencies that cause redundancy in the database. These redundancies are removed by a more strict normal form known as BCNF.
- Example:
- Consider relation R(A, B, C, D, E) A -> BC, CD -> E, B -> D, E -> A
- All possible candidate keys in above relation are {A, E, CD, BC} All attributes are on right sides of all functional dependencies are prime so it is in 3 rd normal form

# BOYCE-CODD NORMAL FORM (BCNF)

- Boyce-Codd Normal Form(BCNF) is an advanced version of 3NF as it contains additional constraints compared to 3NF.
- For a relational table to be in Boyce-Codd normal form, it must satisfy the following rules:
- The table must be in the third normal form.
- For every non-trivial functional dependency X -> Y, X is the superkey of the table. That means X cannot be a non-prime attribute if Y is a prime attribute.
- A superkey is a set of one or more attributes that can uniquely identify a row in a database table.
- Let us take an example of the following <EmployeeProjectLead> table to understand how to normalize the table to the BCNF:

- A relation R is in BCNF if R is in Third Normal Form and for every FD, LHS is
- super key. A relation is in BCNF iff in every non-trivial functional dependency X –
- > Y, X is a super key.
- Example: Relation Student_detail { Stud_id, Stud_Name, City, Zip}
- We find that in the above Student_detail relation, Stud_ID is the key and only prime key attribute. We find that City can be identified by Stud_ID as well as Zipitself. Neither Zip is a superkey nor is City a prime attribute. Additionally, Stud_ID→ Zip → City, so there exists transitive dependency.
- To bring this relation into third normal form, we break the relation into two relations as follows –
- Student_detail { Stud_id, Stud_Name, Zip}
- ZipCodes { Zip, City}
- In the above relationships Stud_id is the super key in the relation Student_detail and Zip is the Super key in the relation ZipCodes

| Employee Code | Project ID | Project Leader |
|---|---|---|
| 101 | P03 | Grey |
| 101 | P01 | Christian |
| 102 | P04 | Hudson |
| 103 | P02 | Petro |
| | | |

The above table satisfies all the normal forms till 3NF, but it violates the rules of BCNF because the candidate key of the above table is {Employee Code, Project ID}.
For the non-trivial functional dependency, Project Leader -> Project ID, Project ID is a prime attribute but Project Leader is a non-prime attribute. This is not allowed in BCNF.

To convert the given table into BCNF, we decompose it into two tables:

**<EmployeeProject>**

| Employee Code | Project ID |
|---|---|
| 101 | P03 |
| 101 | P01 |
| 102 | P04 |
| 103 | P02 |

**<ProjectLead>**

| Project Leader | Project ID |
|---|---|
| Grey | P03 |
| Christian | P01 |
| Hudson | P04 |
| Petro | P02 |

Thus, we've converted the <EmployeeProjectLead> table into BCNF by decomposing it into <EmployeeProject> and <ProjectLead> tables.

- **Example 2:** Find the highest normal form of a relation R(A,B,C,D,E) with FD set as {BC->D, AC->BE, B->E}
- **Step 1:** As we can see, (AC)+ ={A,C,B,E,D} but none of its subset can determine all attribute of relation, So AC will be candidate key. A or C can't be derived from any other attribute of the relation, so there will be only 1 candidate key {AC}.
- **Step 2:** Prime attributes are those attributes that are part of candidate key {A, C} in this example and others will be non-prime {B, D, E} in this example.
- **Step 3:** The relation R is in 1st normal form as a relational DBMS does not allow multi-valued or composite attribute.
- The relation is in 2nd normal form because BC->D is in 2nd normal form (BC is not a proper subset of candidate key AC) and AC->BE is in 2nd normal form (AC is candidate key) and B->E is in 2nd normal form (B is not a proper subset of candidate key AC).

# CONT…

- The relation is not in 3rd normal form because in BC->D (neither BC is a super key nor D is a prime attribute) and in B->E (neither B is a super key nor E is a prime attribute) but to satisfy 3rd normal for, either LHS of an FD should be super key or RHS should be prime attribute. So the highest normal form of relation will be 2nd Normal form.

- For example consider relation R(A, B, C) A -> BC, B -> A and B both are super keys so above relation is in BCNF.

# FOURTH NORMAL FORM

- A relation will be in 4NF if it is in Boyce Codd normal form and has no multi- valued dependency.
- For a dependency A → B, if for a single value of A, multiple values of B exists,
- then the relation will be a multi-valued dependency. In order to denote a multi-valued dependency, "->->" this sign is used.
- Example: Student { Stud_id, Course, Hobby}
- The given STUDENT table is in 3NF, but the COURSE and HOBBY are two independent entity. Hence, there is no relationship between COURSE and HOBBY.
- In the STUDENT relation, a student with STU_ID, 21 contains two courses, Computer and Math and two hobbies, Dancing and Singing. So there is a Multi-valued dependency on STU_ID, which leads to unnecessary repetition of data.
- So to make the above table into 4NF, we can decompose it into two tables:
- Student_course { Stud_id, Course}
- Student_Hobby { Stud_id, Hobby}

## EXAMPLE

| Student-ID | Course | Hobby |
|---|---|---|
| 100 | Science | Dancing |
| 100 | Maths | Singing |
| 101 | C# | Dancing |
| 101 | PHP | Singing |

In this relation Student-ID 1 has thus opted for two courses and has two hobbies. Similarly Student-ID 2 has opted for two courses and has two hobbies. Thus it contains multi-valued dependencies. It is not in 4NF and in order to convert it into 4NF it can be decomposed into two relations:

Now this relation is thus in 4NF. A relation can contain a functional dependency along with a multi-valued dependency also.
So when such a case arises the columns which are functionally dependent are moved to a separate table and the columns which are multi-valued dependent are moved to a separate table. This converts the relation into 4NF.

**Table** R1

| Student-ID | Course |
|---|---|
| 100 | Science |
| 100 | Maths |
| 101 | C# |
| 101 | PHP |

**Table** R2

| Student-ID | Hobby |
|---|---|
| 100 | Dancing |
| 100 | Singing |
| 101 | Dancing |
| 101 | Singing |

# 5TH NORMAL FORM

- A database is in 5NF when there is no join dependency present in the table / database.
- It must be in Fourth Normal Form (4NF).
- It should have no join dependency and also the joining must be lossless.
- When we decompose the given table to remove redundancy in the data and then compose it again to create the original table , we should not lose any data, and the original table should be obtained ,after the decomposition of the table.
- Join dependency for relation R can be stated as
- R=(R1 ⋈ R2 ⋈ R3 ⋈ ……Rn) where R1,R2,R3…..Rn are sub-relation of R and ⋈ is Natural Join Operator.

# FIFTH NORMAL FORM

- o A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless.
- 5NF is satisfied when all the tables are broken into as many tables as possible in order to avoid redundancy.
- o 5NF is also known as Project-join normal form (PJ/NF).
- Example:
- Relation P={Subject, Class, Teacher}

| Subject | Class | Teacher |
|---|---|---|
| Maths | Class 10 | Karthik |
| Math | Class 9 | Yash |
| Math | Class 9 | Yash |
| Math | Class 10 | Yash |
| Science | Class 10 | Yash |

# EXAMPLE

- let's, take **of** Table R which has 3 columns i.e. subject, class, and teacher where each subject can be taught by many teachers in many classes, and a teacher can teach more than 1 subject.
- Here the subject of math is taught by both teachers kartik and yash. Also yash can teach math and science. Yash teaches math to both class 9 and class 10.
- As there is redundancy in data we will decompose it into two tables R1 and R2 such that R1 will have attribute Subject and Class and R2 will have attribute class and teacher.
- **Table R2**

| Class | Teacher |
|---|---|
| class 10 | kartik |
| class 9 | yash |
| class 10 | yash |

**Table R1**

| Subject | Class |
|---|---|
| Math | class 10 |
| Math | class 9 |
| Science | class 10 |

- Here we removed the redundancy in the table by removing the extra tuple with the same values i.e. subject math taught in class 10. This tuple is repeated 2 times in the main table but in table R1 this redundancy is removed.
- Here we removed the redundancy in the table by removing the extra tuple with the same values i.e. yash is teaching for class 10. This tuple is repeated 2 times in the main table but in table R2 this redundancy is removed.
- After combining both tables R1 and R2 we will get as mentioned below:

## Table (R1 ⋈ R2)

| Subject | Class | Teacher |
|---|---|---|
| math | class 9 | yash |
| math | class 10 | kartik |
| math | class 10 | yash |
| science | class 10 | kartik |
| science | class 10 | yash |

Here if we notice the newly composed table from R1 and R2 and the original table, an extra tuple is added that did not exist in the original data, This breaks the second rule of 5NF i.e. non-loss decomposition.

This type of unwanted tuple is known as Spurious tuple.

Here we will decompose the given table in another relation R3 where it will have 2 columns i.e. subject and teacher.

## Table R3

| Subject | Teacher |
|---|---|
| math | yash |
| math | kartik |
| science | yash |

Here the newly decomposed table R3 will have 3 tuples only as the repeated tuple (redundancy ) is not added to the table. yash teaching the subject math is repeated 2 times in main table R but here it will be added only one time resulting in removing the redundancy in the table.

Now if we compose or rejoin the tables R1, R2, and R3 we will get Table (R1 ⋈ R2⋈ R3)

- Now if we see the re-composed table and the original table, there is no loss of data.
- Here all the tables, R1, R2 and R3 had a natural join which resulted in the table R. After the natural join, the original table is retained as it is. There is no loss of the data.
- Given Table R1, R2 and R3 are in the **Fifth Normal Form(5NF)**.

| Subject | Class | Teacher |
|---|---|---|
| math | class 9 | yash |
| math | class 10 | yash |
| math | class 10 | kartik |
| science | class 10 | yash |

# USES OF FIFTH NORMAL FORM(5NF)

- 5NF ensures that there will be no redundancy present in the database.
- Removing the redundancy in the database helps the data to remain more optimized and easy to perform database actions.
- It also ensures that there will be non-lossy decomposition only which will result in data consistency and data integrity.
- As data redundancy and anomalies are removed, the database performance gets enhanced.

# LIMITATION OF FIFTH NORMAL FORM(5NF)

- One of the biggest limitations**is** of 5Nf is the complexity of the database. Due to 5Nf large number of tables and relation gets created which eventually increases the complexity of the database.
- Slow execution due to large number of tables.
- The cost of implementation of 5NF is also high as it increases the complexity of the database.
- As 5NF increases the complexity of the databse, it is less frequently used in the indusrty. Complexity is the reason for less use of 5NF.

# ADVANTAGES OF NORMALIZATION

- Normalization helps to minimize data redundancy.
- Greater overall database organization.
- Data consistency within the database.
- Much more flexible database design.
- Enforces the concept of relational integrity.

# DISADVANTAGES OF NORMALIZATION

- You cannot start building the database before knowing what the user needs.
- The performance degrades when normalizing the relations to higher normal forms, i.e., 4NF, 5NF.
- It is very time-consuming and difficult to normalize relations of a higher degree.
- Careless decomposition may lead to a bad database design, leading to serious problems